# Computer-Assisted Formalization of Mathematics

## Week 08 - Functors, applicatives, and monads

Jaume de Dios Pont

New York University

Week 08

Functors $\longleftarrow$ Applicatives $\longleftarrow$ Monads

# Functors

For a type constructor $F$, such as `Option` or `List`, a **functor** provides:

$$\text{map} : (\alpha \to \beta) \to F\,\alpha \to F\,\beta$$

```
#eval Functor.map (fun n : Nat => n + 1) (some 4) -- some 5
#eval Functor.map (fun n : Nat => n + 1) none     -- none
```

**Notation:** `g <$> x` means `Functor.map g x`.

# Functors

For a type constructor $F$, such as `Option` or `List`, a **functor** provides:

$$\text{map} : (\alpha \to \beta) \to F\,\alpha \to F\,\beta$$

```
#eval Functor.map (fun n : Nat => n + 1) (some 4) -- some 5
#eval Functor.map (fun n : Nat => n + 1) none     -- none
```

**Notation:** `g <$> x` means `Functor.map g x`.

`LawfulFunctor F` proves:

| | |
|---|---|
| `id_map` | map id $x = x$ |
| `comp_map` | map $(h \circ g)\ x =$ map $h$ (map $g$ $x$) |

# Applicative functors

An **applicative functor** combines values in $F$, such as two optional numbers.

$$\text{pure} : \alpha \to F\,\alpha$$
$$\text{seq} : F\,(\alpha \to \beta) \to F\,\alpha \to F\,\beta$$

```
#eval Nat.add <$> some 3 <*> some 4 -- some 7
#eval Nat.add <$> some 3 <*> none   -- none
```

**Notation:** `f <*> arg` means seq *f arg*.

An applicative is a functor by defining

$$\text{map}\ g\ x := \text{seq}\ (\text{pure}\ g)\ x.$$

# Applicative functors in Lean

An **applicative functor** combines values in $F$, such as two optional numbers.

$$\text{pure} : \alpha \to F\,\alpha$$
$$\text{seq} : F\,(\alpha \to \beta) \to (\text{Unit} \to F\,\alpha) \to F\,\beta$$

```
#eval Nat.add <$> some 3 <*> some 4 -- some 7
#eval Nat.add <$> some 3 <*> none   -- none
```

**Notation:** `f <*> arg` means seq $f$ (fun () => $arg$).

An applicative is a functor by defining

$$\text{map}\ g\ x := \text{seq}\ (\text{pure}\ g)\ (\text{fun}\ _ \Rightarrow x).$$

Lean delays the second argument so an instance can stop on failure.

# Applicative laws

`LawfulApplicative F` requires, on top of the functor laws:

| | |
|---|---|
| `pure_seq` | pure $g$ <*> $x = g$ <$> $x$ |
| `map_pure` | $g$ <$> pure $x =$ pure $(g\ x)$ |

# Applicative laws

`LawfulApplicative F` requires, on top of the functor laws:

| | |
|---|---|
| `pure_seq` | pure $g$ <*> $x = g$ <$> $x$ |
| `map_pure` | $g$ <$> pure $x =$ pure $(g\ x)$ |
| `seq_pure` | $u$ <*> pure $x =$ (fun $g$ => $g\ x$) <$> $u$ |
| `seq_assoc` | $u$ <*> $(v$ <*> $w) = ((\circ)$ <$> $u)$ <*> $v$ <*> $w$ |

# Monads (reminder)

A **monad** lets the next computation depend on the previous result:

$$\text{bind} : M\,\alpha \to (\alpha \to M\,\beta) \to M\,\beta$$

**Notation:** `action >>= next`. In do, `let x ← action` uses bind.

```
def followIndex (xs : List Nat) : Option Nat := do
  let i ← xs[0]?
  xs[i]?
```

# Monads (reminder)

A **monad** lets the next computation depend on the previous result:

$$\text{bind} : M\,\alpha \to (\alpha \to M\,\beta) \to M\,\beta$$

**Notation:** `action >>= next`. In do, `let x ← action` uses bind.

```
def followIndex (xs : List Nat) : Option Nat := do
  let i ← xs[0]?
  xs[i]?
```

A monad is applicative: use its `pure` and define application by

```
seq f arg := do
  let g ← f
  let x ← arg ()
  pure (g x)
```

# Monad laws

`LawfulMonad M` requires lawful applicative behavior and:

| | |
|---|---|
| `pure_bind` | pure $a$ >>= $f = f\ a$ |
| `bind_pure` | $x$ >>= pure $= x$ |
| `bind_assoc` | $(x$ >>= $f)$ >>= $g = x$ >>= (fun $a$ => $f\ a$ >>= $g)$ |

# Monad laws

`LawfulMonad M` requires lawful applicative behavior and:

| | |
|---|---|
| `pure_bind` | pure $a$ >>= $f = f\ a$ |
| `bind_pure` | $x$ >>= pure $= x$ |
| `bind_assoc` | $(x$ >>= $f)$ >>= $g = x$ >>= (fun $a$ => $f\ a$ >>= $g)$ |
| `bind_pure_comp` | $x$ >>= (fun $a$ => pure $(g\ a)) = g$ <$> $x$ |
| `bind_map` | $u$ >>= (fun $g$ => $g$ <$> $x) = u$ <*> $x$ |

# When to use each of them?

**Functor**: apply a pure function to one effectful value.

```
(· + 1) <$> xs[0]?
```

# When to use each of them?

**Functor**: apply a pure function to one effectful value.

```
(· + 1) <$> xs[0]?
```

**Applicative**: apply a pure function to several effectful values.

```
(· + ·) <$> xs[0]? <*> xs[2]?
```

# When to use each of them?

**Functor**: apply a pure function to one effectful value.

```
(· + 1) <$> xs[0]?
```

**Applicative**: apply a pure function to several effectful values.

```
(· + ·) <$> xs[0]? <*> xs[2]?
```

**Monad**: use a result to choose the next computation.

```
xs[0]? >>= fun i => xs[i]?
```

# Monad transformers

A monad transformer takes a monad and produces another monad:

$$T : \text{monads} \to \text{monads}, \qquad m \mapsto T\ m$$

Its `pure` and `bind` use the operations of $m$.

# Monad transformers

A monad transformer takes a monad and produces another monad:

$$T : \text{monads} \to \text{monads}, \qquad m \mapsto T\ m$$

Its `pure` and `bind` use the operations of $m$.

**Example: state with possible failure**

$$\text{StateT Nat Option } \alpha = \text{Nat} \to \text{Option } (\alpha \times \text{Nat})$$

```
def spend : StateT Nat Option Unit := do
  let n ← get
  if n == 0 then failure else set (n - 1)
#eval spend.run 1 -- some ((), 0)
#eval spend.run 0 -- none
```

**Available:** `OptionT`, `ExceptT`, `StateT`, `ReaderT`.

# Imperative programming in Lean

# Control-flow syntax in do

```
def countdowns [Monad m] (emit : Nat → m Unit)
    (starts : List Nat) : m Unit := do
  for start in starts do
    let mut n := start
    while n > 0 do
      if n % 2 == 0 then emit n
      n := n - 1
```

# Control-flow syntax in do

```
def countdowns [Monad m] (emit : Nat → m Unit)
    (starts : List Nat) : m Unit := do
  for start in starts do
    let mut n := start
    while n > 0 do
      if n % 2 == 0 then emit n
      n := n - 1
```

A condition can itself perform effects: `if (← test) then` or `while (← test) do`.

**Also supported**: `break`, `continue`, `return`.

# A `for` loop as monadic recursion

```
variable {m : Type → Type} [Monad m]
  (f : Nat → m Nat) (xs : List Nat)
```

**With `for`**

```
def sumFor : m Nat := do
  let mut s := 0
  for x in xs do
    let n ← f x
    s := s + n
  pure s
```

# A `for` loop as monadic recursion

```
variable {m : Type → Type} [Monad m]
  (f : Nat → m Nat) (xs : List Nat)
```

**With `for`**

```
def sumFor : m Nat := do
  let mut s := 0
  for x in xs do
    let n ← f x
    s := s + n
  pure s
```

**After expanding `ForIn`**

```
def sumBind : m Nat :=
  let rec loop :
      List Nat → Nat → m Nat
    | [], s => pure s
    | x :: rest, s =>
        f x >>= fun n =>
          loop rest (s + n)
  loop xs 0
```

The mutable sum becomes an argument to `loop`, updated after bind supplies `n`.